

Solutions last updated: Thursday, October 23, 2025

PRINT Your Name: _____

PRINT Your Student ID: _____

PRINT Student name to your left: _____

PRINT Student name to your right: _____

You have 110 minutes. There are 6 questions of varying credit. (100 points total)

Question:	1	2	3	4	5	6	Total
Points:	19	16	19	19	17	10	100

We reserve the right to deduct points for failing to follow the marking directions below:

For questions with **circular bubbles**, you may select only one choice.

☐ Unselected option (Completely unfilled)

☐ Don't do this (it will be graded as incorrect!)

☒ Only one selected option (completely filled)

For questions with **square checkboxes**, you may select one or more choices.

☐ You can select

☐ multiple squares

☒ Don't do this (it will be graded as incorrect!)

Anything you write outside the answer boxes or you ~~cross-out~~ will not be graded. If you write multiple answers, your answer is ambiguous, or the bubble/checkbox is not entirely filled in, we will grade the worst interpretation.

Read the honor code below and sign your name.

By signing below, I affirm that all work on this exam is my own work. I have not referenced any disallowed materials, nor collaborated with anyone else on this exam. I understand that if I cheat on the exam, I may face the penalty of an "F" grade and a referral to the Center for Student Conduct.
--

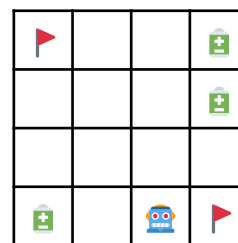
SIGN your name: _____

Q1 Search: Delivery Drone Dilemma

(19 points)

Consider a delivery drone moving on a $N \times N$ grid.

- At each time step, the drone must move to an adjacent square (up, down, left, right). All actions cost 1.
- There are k fixed destination squares on the grid. The drone needs to visit all k destinations at least once.
- The drone starts with a battery charge of C units, and must always have at least 1 unit of charge at every time step. The drone loses 1 unit of charge per action.
- There are p power cells on the grid (with fixed locations). When the drone visits a power cell, its battery charge resets to C units, and the power cell cannot be used again.



Sample grid with $N = 4$,
 $k = 2$ destinations 🚩,
 $p = 3$ power cells 🔋,
 and drone 🚁.

Q1.1 (2 points) What is the maximum branching factor for this problem? Write an integer in the box.

4

Solution: 4, because at most the drone can move to any of the four adjacent squares.

Q1.2 (3 points) What is the size of the smallest state space representation for this problem?

Your answer is the product of all selected choices, e.g. if you select N^2 and C , your answer is N^2C .

☐ A N

☐ C p

☐ E k

☒ G C

☒ B N^2

☒ D 2^p

☒ F 2^k

☐ H C^2

Solution:

The drone's position can be one of N^2 squares.

At any point in time, any combination of the p power cells and k destination squares will be visited, so we need one Boolean per power cell and one Boolean per destination square.

The drone's current battery charge will be one of C different values (it can never be empty).

This gives a size of: CN^22^{p+k}

(Question 1 continued...)

Q1.3 (2 points) Select all algorithms that are guaranteed to find the lowest-cost solution to this problem (assuming one exists).

Note: “Lowest-cost” refers to each action costing 1, which is separate from the battery rules.

- | | |
|--|--|
| ☐ A DFS tree search | ☒ E UCS tree search |
| ☐ B DFS graph search | ☒ F UCS graph search |
| ☒ C BFS tree search | ☐ G None of the above |
| ☒ D BFS graph search | |

Solution:

DFS is not guaranteed to find the lowest-cost solution.

BFS will find the lowest-cost solution, since all actions cost 1.

UCS will find the lowest-cost solution in general.

Both tree and graph search work, because the solution is finite-length (the length of the solution is limited by the battery charge, so it can't be infinite).

Q1.4 (3 points) Suppose we are running BFS tree search, and there are 7 search tree nodes on the fringe.

After we dequeue one node, and enqueue all of its successors, how many nodes could possibly be on the fringe? Select all that apply.

- ☐ A 5 ☒ B 6 ☒ C 7 ☒ D 8 ☒ E 9 ☒ F 10 ☐ G 11 ☐ H 12

Solution:

5 is false. You would have 6 nodes left after popping one node, and there's no way to enqueue -1 successors to reach 5 nodes.

6 is true. Suppose the node we popped is a partial plan where we have 1 battery charge unit left, and none of the adjacent squares contain a power cell. Then this node has zero successors.

7 is true. Suppose the node we popped is a partial plan where we have 1 battery charge unit left, and only one adjacent square contains a power cell. Then this node has 1 successor state.

8 is true. Suppose the node we popped is a partial plan where we have lots of battery charge left, and we are in a corner square (with two adjacent squares). Then this node has 2 successor states.

9 is true. Suppose the node we popped is a partial plan where we have lots of battery charge left, and we are on an edge square (with three adjacent squares). Then this node has 3 successor states.

10 is true. Suppose the node we popped is a partial plan where we have lots of battery charge left, and we are on a center square (with four adjacent squares). Then this node has 4 successor states.

11 and 12 are false. We have a maximum branching factor of 4, meaning we cannot add more than 4 successor states.

(Question 1 continued...)

Q1.5 (5 points) Select all admissible heuristics.

- ☒ A Manhattan distance to the closest unvisited destination, or 0 if all destinations are visited.
- ☒ B Euclidean distance to the furthest unvisited destination, or 0 if all destinations are visited.
- ☐ C Number of unused power cells.
- ☒ D Number of unvisited destinations.
- ☐ E Current battery charge.
- ☐ F None of the above

Solution: Manhattan distance to the closest unvisited destination (0 if all destinations are visited) works because the drone has to get to at least the closest unvisited destination before reaching the goal state. Since the drone can only follow the cells in the grid, it will need to take at least Manhattan distance time steps. So this is admissible.

Euclidean distance to the furthest unvisited destination (0 if all destinations are visited) works because the drone has to get to all the unvisited destinations, including the furthest unvisited destinations, before reaching the goal state. Euclidean distance $<$ the number of cells in the grid the drone needs to move through to reach the furthest unvisited destination, so this is admissible.

There is no constraint stating that the number of unused power cells at the goal state has to be 0, so this is not an admissible heuristic.

Since each destination is a different cell, the drone will need to move through at least as many cells as there are destinations left to visit. So this is admissible.

At the goal state, the current battery charge can be any value ranging from 0 to $C - 1$, so this is not an admissible heuristic.

Q1.6 (2 points) For this subpart, we want to find an *energy-maximizing solution*, which is a solution (not necessarily lowest-cost) that maximizes the battery charge when the drone reaches a goal state.

Select all algorithms that are guaranteed to output an energy-maximizing solution.

- ☐ A BFS graph search, with no modifications.
- ☒ B BFS graph search, modified to continue outputting solutions until the entire fringe is empty.
- ☐ C None of the above

Solution:

(A) is false. Consider a grid with 1 destination, where the least-cost solution reaches the destination with low battery. But, an alternative solution that visits an extra battery reaches the destination with more battery.

(B) is true, because this modified algorithm will output all solutions (even ones that are not least-cost), and we can check them all to see which is the energy-maximizing solution.

(Question 1 continued...)

Q1.7 (2 points) For this subpart only, we modify the search problem: The drone must finish visiting all destination squares within T actions.

Do we need to change the state space representation to solve this modified problem?

☒ A Yes

☐ B No

Solution:

The state space representation includes the drone's position, Booleans per power cell, Booleans per destination square, and the drone's current charge. From this information, given a state, we cannot infer how many actions have been taken so far to reach that state.

Note: Since each battery fully resets our current battery charge, there is no way to compute the number of actions taken from the number of batteries picked up and the current battery charge.

Therefore, we must add additional information to our state space representation to handle the modified search problem.

Q2 CSPs: Confused Subway Planners

(16 points)

Pacman is designing the new BART train schedule for the Oski Line:

- There are 4 variables, corresponding to 4 trains (numbered 101 through 104).
- The values are the possible arrival times at the Berkeley Station: 3:00pm, 3:05pm, ..., 3:55pm, 4:00pm.

The Oski Line's train schedule must satisfy these rules:

1. There must be at least 10 minutes between two trains arriving, e.g. if a train arrives at 3:10pm, then no train can arrive at 3:05pm, 3:10pm, or 3:15pm.
2. Train 103 must arrive earlier than Train 102.
3. A train must arrive at the Berkeley Station at 3:30pm.
4. Train 101 must arrive at the Berkeley Station between 3:00pm and 3:20pm (inclusive).

Q2.1 (2 points) If each rule is represented using a single constraint, which rules must be represented with a **higher-order** constraint? Select all that apply.

☒ A 1

☐ B 2

☒ C 3

☐ D 4

Solution: To enforce rules 1 and 3 we must check all trains. Since we need to use a single constraint, these are all higher-order constraints.

Q2.2 (2 points) If each rule is represented using a single constraint, which rules can be represented with a **unary** constraint? Select all that apply.

☐ A 1

☐ B 2

☐ C 3

☒ D 4

Solution: Rule 4 only involves Train 101, so it's unary.

Consider the partial assignment (with only Train 104 assigned) and domains below:

Train 101: {3:15pm, 3:20pm}

Train 102: {3:00pm, 3:20pm, 3:45pm, 3:50pm, 3:55pm}

Train 103: {3:20pm, 3:30pm, 3:40pm}

Train 104: 3:30pm

Subparts Q2.3 and Q2.4 are independent.

Q2.3 (2 points) Which of these arcs is already consistent, without removing values from any domains?

☐ A (Train 102 $\rightarrow$ Train 103) only.

☐ C Both are consistent.

☒ B (Train 103 $\rightarrow$ Train 102) only.

☐ D Neither are consistent.

Solution: (Train 102 $\rightarrow$ Train 103) is not consistent because if Train 102 were assigned to 3:00pm or 3:20pm, there would be no possible assignment for Train 103 from the domain (since Train 103 has to arrive before 102 from rule 2).

(Train 103 $\rightarrow$ Train 102) is consistent since for any assignment of Train 103, we can assign Train 102 to 3:55pm, which successfully satisfies rules 1 and 2.

(Question 2 continued...)

Q2.4 (2 points) Suppose we assign Train 103 to 3:40pm and run forward checking.

Select all values that **remain** in the domain of Train 102.

- ☐ A 3:00pm ☐ B 3:20pm ☐ C 3:45pm ☒ D 3:50pm ☒ E 3:55pm

Solution: Checking rule 1, we can prune the value 3:45pm from the domain of Train 102. Checking rule 2, we can prune 3:00pm and 3:20pm. Rule 4 is irrelevant here since it does not involve Train 102, so our final domain is {3:50pm, 3:55pm}.

Now, suppose we modify the CSP, such that it outputs train arrival times at both the Berkeley Station and the Ashby Station (the next station down the line).

- All four trains are traveling down the line, from Berkeley to Ashby.
- Every train takes exactly 10 minutes to travel from Berkeley to Ashby. 4:05pm and 4:10pm are added to the domain of every variable.
- The four rules from earlier still apply to Berkeley arrivals.

Q2.5 (2 points) How many total variables are needed to model this new CSP, such that it outputs train arrival times at both stations?

- ☒ A 2 ☐ B 6 ☒ C 8 ☐ D 12 ☐ E 16

Solution: 8 variables. We need variables representing arrival of each train at each station.

We want to add as few unary and binary constraints as possible to correctly model this new CSP.

Q2.6 (2 points) How many additional unary constraints do we need to add?

- ☒ A 0 ☐ B 1 ☐ C 2 ☐ D 3 ☐ E 4

Solution: No new unary constraints are needed, because the binary constraints below are sufficient to output the arrival times at Ashby Station.

Q2.7 (2 points) How many additional binary constraints do we need to add?

- ☐ A 0 ☐ B 1 ☐ C 2 ☐ D 3 ☒ E 4

Solution: Four new binary constraints are needed.

“If Train 101 arrives at Berkeley at time x , then it must arrive at Ashby at time $x + 10$.”

We need one of these constraints per train, and there are 4 trains.

(Question 2 continued...)

Q2.8 (2 points) Does the original CSP (Berkeley only) or the modified CSP (Berkeley and Ashby) have more possible solutions?

- ☐ A Original, because it has fewer constraints. ☒ C Both have the same number of solutions.
☐ B Modified, because it has more constraints. ☐ D Not enough information.

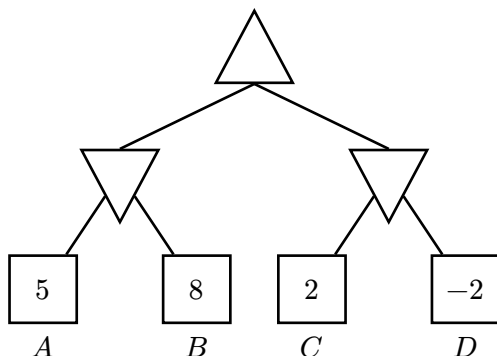
Solution: Each solution to the original CSP corresponds to exactly one solution to the modified CSP (assigning Ashby arrivals exactly 10 minutes later).

Q3 Games: The Battle of Pac-Man

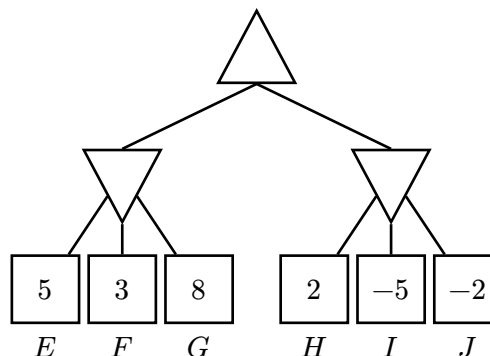
(19 points)

Pacman (maximizer) and Blinky (minimizer) are pirates on rival ships, and they are engaged in a battle!

Pacman first solves a minimax game tree, assuming Blinky has 2 possible actions. Then, Pacman learns that Blinky actually has 3 possible actions, and solves an updated minimax game tree.



Tree 1: before Blinky's extra action



Tree 2: after Blinky's extra action

Q3.1 (1 point) What is the root value for Tree 1?

- ☒ A 5 ☐ B 8 ☐ C 2 ☐ D -2

Solution: 5 – On Blinky's turn, he will take the minimum of (5, 8) and (2, -2). Pacman will then take the maximum of (5, 2) to get 5.

Q3.2 (1 point) What is the root value for Tree 2?

- ☐ A 5 ☒ B 3 ☐ C 8 ☐ D 2 ☐ E -5 ☐ F -2

Solution: 3 – On Blinky's turn, he will take the minimum of (5, 3, 8) and (2, -5, -2). Pacman will then take the maximum of (3, -5) to get 3.

Consider running alpha-beta pruning for Tree 2, visiting nodes from left to right.

Q3.3 (3 points) Which leaf nodes are **not visited** due to pruning? Select all that apply.

- ☐ E ☐ F ☐ G ☐ H ☒ I ☒ J

Solution: E, F and G must be visited to determine the minimum value of the left subtree=3, but after visiting H we can prune I and J because H is already less than 3.

(Question 3 continued...)

Q3.4 (2 points) For this subpart, suppose the terminal nodes in Tree 2 can take on any values.
Can we ever prune node H ?

☐ A Yes

☒ B No

Solution: Visiting H is required because otherwise, you have no information bounding the right minimizer node, so you don't know which action will be taken at the root.

For the rest of the question, consider an arbitrary game tree, i.e. not necessarily the trees above.

- The root node is a maximizer node.
- The tree can be any depth, with any values at the terminal nodes.
- When we add an extra action, the original parts of the tree are unchanged.

For each subpart, we want to compare the value at the root *after* adding an extra action, compared to the value at the root *before* adding the extra action.

For each subpart, select all that apply.

In Q3.5 and Q3.6, consider a **minimax** game tree.

Q3.5 (3 points) If we add an extra action to every **minimizer** node, what could happen to the root value?

☐ A The value could increase.

☒ C The value could stay the same.

☒ B The value could decrease.

☐ D None of the above

Solution:

Both the maximizer and minimizer agents play optimally. Adding a new possible action to the minimizer can lead to two scenarios:

- 1) The minimizer will use the new action, resulting in a lower value at the minimizer.
- 2) The minimizer will use the original action, resulting in the same value at the minimizer.

In either case, any maximizer value (including the root value) can only decrease or stay the same.

Q3.6 (3 points) If we add an extra action to every **maximizer** node, what could happen to the root value?

☒ A The value could increase.

☒ C The value could stay the same.

☐ B The value could decrease.

☐ D None of the above

Solution: The reasoning is similar as the previous subpart.

Each maximizer's value will either stay the same (from using the original action), or increase (from using a new action).

(Question 3 continued...)

In Q3.7 and Q3.8, consider a **expectimax** game tree (with only maximizer, chance, and terminal nodes).

Q3.7 (3 points) If we add an extra action to every **chance** node, what could happen to the root value?

- ☒ A The value could increase.
- ☒ C The value could stay the same.
- ☒ B The value could decrease.
- ☒ D None of the above

Solution: With an extra action added to each chance node, the value at those chance nodes could go up, down, or stay the same depending on the utility of the added action. So, the expectimax value at the root could go up, down, or stay the same.

Q3.8 (3 points) If we add an extra action to every **maximizer** node, what could happen to the root value?

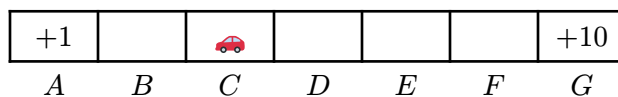
- ☒ A The value could increase.
- ☒ C The value could stay the same.
- ☒ B The value could decrease.
- ☒ D None of the above

Solution: From the solution in 3.6, we know the value of the maximizer node will never decrease. Since the chance nodes are the weighted sum of the maximizer nodes, it will only increase or stay the same. Therefore, the root node will follow, staying the same or increasing for any arbitrary tree.

Note: The solution is agnostic to the sequencing of the layers. Having multiple chance layers in a row can be collapsed to a single chance layer, and having multiple max or min layers in a row can be collapsed to a single max or min function.

Q4 MDPs: Most Drivable Prius**(19 points)**

Consider a car agent in the Gridworld below.



Remember that in Gridworld, when you are in the +1 or +10 square, the only action available is “Exit,” and you must take the “Exit” action to earn the reward. In all other squares, the available actions are “Left” and “Right”.

All actions succeed with 100% probability, $\gamma = 0.5$, and there is 0 living reward.

Q4.1 (2 points) When running value iteration, what is the first iteration k where $V_k(C)$ is nonzero?

- ☐ Ⓐ 0 ☐ Ⓑ 1 ☐ Ⓒ 2 ☒ Ⓓ 3 ☐ Ⓔ 4 ☐ Ⓕ 5 ☐ Ⓖ 6

Solution: On the first iteration, $V_1(A)$ will become nonzero because of the +1 reward when you take the “Exit” action from G. On the second iteration, $V_2(B)$ will become nonzero because of the nonzero value at A. On the third iteration, $V_3(C)$ will become nonzero because of the nonzero value at B.

(Question 4 continued...)

Q4.2 (5 points) After running value iteration to convergence, what are the optimal values $V^*(s)$ and optimal policies $\pi^*(s)$ at each of the states? Some blanks are filled in for you.

Write a number in each of the top 5 boxes. Write “Left” or “Right” in each of the bottom 5 boxes.

State s :	A	B	C	D	E	F	G
$V^*(s)$	1						10
$\pi^*(s)$	Exit						Exit

Solution:

State s :	A	B	C	D	E	F	G
$V^*(s)$	1	0.5	0.625	1.25	2.5	5	10
$\pi^*(s)$	Exit	Left	Right	Right	Right	Right	Exit

These can be derived without running value iteration, and by using intuition instead.

We know that the optimal policy must be either:

- Go all the way left and exit at the +1 square.
- Go all the way right and exit at the +10 square.

Any other policy would yield a reward of 0, and be sub-optimal.

So, at each state, we just need to compute the expected discounted reward of each policy, and see which one yields higher discounted reward.

State s :	B	C	D	E	F
Left:	0.5	0.25	0.125	0.0625	0.03125
Right:	0.3125	0.625	1.25	2.5	5

Consider a modification called *potential-based reward shaping*, where we replace the reward function R with a modified reward function R' :

$$R'(s, a, s') = R(s, a, s') + [\gamma f(s') - f(s)]$$

where f is a *potential function* that maps states to values.

For the Gridworld above, suppose our potential function $f(s)$ is the negative distance between state s and the +10 square, as shown below:

	+1						+10
State s :	A	B	C	D	E	F	G
$f(s)$:	-6	-5	-4	-3	-2	-1	0

(Question 4 continued...)

Note: For the terminal state after taking the “Exit” action, $f(\text{terminal state}) = 0$.

Given the $f(s)$ values above and 0 living reward, we can derive these values for $R'(s, a, s')$:

$$\begin{aligned} R'(B, \text{Left}, A) &= 2 & R'(C, \text{Left}, B) &= 1.5 & R'(D, \text{Left}, C) &= 1 \\ R'(B, \text{Right}, C) &= 3 & R'(C, \text{Right}, D) &= 2.5 & R'(D, \text{Right}, E) &= 2 \end{aligned}$$

We can now run value iteration with this modified reward function:

$$V_{k+1}'(s) = \max_a \sum_{s'} T(s, a, s') \left[\underbrace{R(s, a, s') + [\gamma f(s') - f(s)]}_{R'(s, a, s')} + \gamma V_k'(s') \right]$$

Q4.3 (3 points) Let $V_0'(s) = 0$ for all states. Fill in $V_1'(s)$ for the states below. Write a number in each box.

State s :	B	C	D
$V_1'(s)$			

Solution:

Here are all the $V_1'(s)$ values, though you only needed to write three of them on the exam:

State s :	A	B	C	D	E	F	G
$V_1'(s)$	7	3	2.5	2	1.5	1	10

Here's how you'd derive them:

$$\begin{aligned} V_1'(B) &= \max_a \left[\begin{array}{l} \text{Left: } R'(B, \text{Left}, A) + \gamma V_0'(A) = 2 + 0.5(0) = 2 \\ \text{Right: } R'(B, \text{Right}, C) + \gamma V_0'(C) = 3 + 0.5(0) = 3 \end{array} \right] = 3 \\ V_1'(C) &= \max_a \left[\begin{array}{l} \text{Left: } R'(C, \text{Left}, B) + \gamma V_0'(A) = 1.5 + 0.5(0) = 1.5 \\ \text{Right: } R'(C, \text{Right}, D) + \gamma V_0'(C) = 2.5 + 0.5(0) = 2.5 \end{array} \right] = 2.5 \\ V_1'(D) &= \max_a \left[\begin{array}{l} \text{Left: } R'(D, \text{Left}, C) + \gamma V_0'(A) = 1 + 0.5(0) = 1 \\ \text{Right: } R'(D, \text{Right}, E) + \gamma V_0'(C) = 2 + 0.5(0) = 2 \end{array} \right] = 2 \end{aligned}$$

(Question 4 continued...)

Q4.4 (4 points) What is the relationship between $V'^*(s)$ and $V^*(s)$?

Write an expression using any of $V^*(s)$, γ , $f(s)$, $T(s, a, s')$, $R(s, a, s')$, and any mathematical operators (you may not need all of these).

Hint: Value iteration with the modified reward function converges to these values:

State s :	A	B	C	D	E	F	G
$V'^*(s)$	7	5.5	4.625	4.25	4.5	6	10

$$V'^*(s) = V^*(s) - f(s)$$

Solution: By inspection of example:

In a previous subpart, we solved for V^s in this Gridworld:

State s :	A	B	C	D	E	F	G
$V^*(s)$	1	0.5	0.625	1.25	2.5	5	10

By inspection, we can see that the values for states A through G differ by 6, 5, 4, 3, 2, 1, and 0, respectively. This is exactly the potential function $f(s)$ at each state, which gives us the answer.

Alternate solution, by proof:

This proof shows that the equation holds for any MDP, not just the Gridworld example. You did not need to do this proof on the exam to solve this question.

First, note that the value of a function is the sum of discounted rewards (where r_i is the reward for the i th action taken):

$$V^*(s) = r_0 + \gamma r_1 + \gamma^2 r_2 + \dots \gamma^\infty r_\infty$$

Now, plug in the modified reward function $r_i = r + \gamma f_{i+1} - f_i$ (where f_i is the potential function at the i th state):

$$\begin{aligned}
 V'^*(s) &= \underbrace{r_0 + (\gamma f_1 - f_0)}_{\text{modified } r_0} + \underbrace{\gamma(r_1 + (\gamma f_2 - f_1))}_{\text{modified } r_1} + \underbrace{\gamma^2(r_2 + (\gamma f_3 - f_2))}_{\text{modified } r_2} + \dots \\
 &= \sum_{i=0}^{\infty} \gamma^i r_i + \sum_{i=0}^{\infty} \gamma^i (\gamma f_{i+1} - f_i) \\
 &= \sum_{i=0}^{\infty} \gamma^i r_i + \underbrace{\cancel{\gamma f_1} - f_0 + \gamma^2 \cancel{f_2} - \cancel{\gamma f_1} + \gamma^3 \cancel{f_3} - \gamma^2 \cancel{f_2} + \dots}_{\text{telescoping sum with cancellations}} \\
 &= \sum_{i=0}^{\infty} \gamma^i r_i - f_0 \\
 &= V^*(s) - f_0
 \end{aligned}$$

After rearranging terms, the first summation is exactly equal to $V^*(s)$.

The second summation is a telescoping sum, so all of its terms cancel out except $-f_0$.

(Question 4 continued...)

Q4.5 (3 points) We extract a policy using the following equation:

$$\pi'^*(s) = \operatorname{argmax}_a \sum_{s'} T(s, a, s') \left[\underbrace{R(s, a, s') + [\gamma f(s') - f(s)]}_{R'(s, a, s')} + \gamma V'^*(s') \right]$$

What is the policy extracted from the optimal values $V'^*(s)$? Some blanks are filled in for you. Write “Left” or “Right” in each box.

Math hints: $0.5(4.625) = 2.3125$, and $0.5(4.25) = 2.125$.

State s :	A	B	C	D	E	F	G
$\pi'^*(s)$	Exit				Right	Right	Exit

Solution:

State s :	A	B	C	D	E	F	G
$\pi'^*(s)$	Exit	Left	Right	Right	Right	Right	Exit

$\pi'^*(s) = \pi^*(s)$ following the solution to 4.6, but we can also calculate the optimal policy following the given equation as below:

$$\begin{aligned} \pi'_1(B) &= \operatorname{argmax}_a \left[\begin{array}{l} \text{Left: } R'(B, \text{Left}, A) + \gamma V'^*(A) = 2 + 0.5(7) = 5.5 \\ \text{Right: } R'(B, \text{Right}, C) + \gamma V'^*(C) = 3 + 0.5(4.625) = 5.3125 \end{array} \right] \\ &= \text{Left} \\ \pi'_1(C) &= \operatorname{argmax}_a \left[\begin{array}{l} \text{Left: } R'(C, \text{Left}, B) + \gamma V'^*(B) = 1.5 + 0.5(5.5) = 4.25 \\ \text{Right: } R'(C, \text{Right}, D) + \gamma V'^*(D) = 2.5 + 0.5(4.25) = 4.625 \end{array} \right] \\ &= \text{Right} \\ \pi'_1(D) &= \operatorname{argmax}_a \left[\begin{array}{l} \text{Left: } R'(D, \text{Left}, C) + \gamma V'^*(C) = 1 + 0.5(4.625) = 3.3125 \\ \text{Right: } R'(D, \text{Right}, E) + \gamma V'^*(E) = 2 + 0.5(4.5) = 4.25 \end{array} \right] \\ &= \text{Right} \end{aligned}$$

(Question 4 continued...)

Q4.6 (2 points) Consider any arbitrary MDP and potential function (not necessarily the ones above). We compute the optimal policy once using R , then again using R' .

The optimal policy computed using R is _____ the same as the optimal policy computed using R' .

Hint: Take your expression from Q4.4, and plug it into the equation from Q4.5.

☒ A Always

☐ B Sometimes

☐ C Never

Solution:

Following the hint, we plug $V'^*(s) = V^*(s) - f(s)$ into the policy extraction equation, and simplify:

$$\begin{aligned}\pi'^*(s) &= \operatorname{argmax}_a \sum_{s'} T(s, a, s') \left[\underbrace{R(s, a, s') + [\gamma f(s') - f(s)]}_{R'(s, a, s')} + \gamma V'^*(s') \right] \\ &= \operatorname{argmax}_a \sum_{s'} T(s, a, s') \left[\underbrace{R(s, a, s') + [\gamma f(s') - f(s)]}_{R'(s, a, s')} + \underbrace{\gamma (V^*(s') - f(s'))}_{\text{plugged this in}} \right] \\ &= \operatorname{argmax}_a \sum_{s'} T(s, a, s') \left[\underbrace{R(s, a, s') + [\cancel{\gamma f(s')} - f(s)]}_{R'(s, a, s')} + \gamma V^*(s') \cancel{- \gamma f(s')} \right] \\ &= \operatorname{argmax}_a \left(\sum_{s'} T(s, a, s') [R(s, a, s') - f(s) + \gamma V^*(s')] \right) \\ &= \operatorname{argmax}_a \left(-f(s) + \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')] \right) \\ &= \operatorname{argmax}_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')] \\ &= \pi^*(s)\end{aligned}$$

In the third-to-last line, we moved the $-f(s)$ term out of the summation, because the summation is a weighted average over different s' , and the term $-f(s)$ does not depend on s' .

In other words, each term of the summation contributes $T(s, a, s') \cdot -f(s)$ to the overall sum. When summing across all s' , the transition probabilities sum to 1, so this simplifies to $-f(s)$.

In the second-to-last line, we removed the $-f(s)$ term from the argmax entirely, because the argmax is over actions a , and adding a constant $-f(s)$ does not affect the argmax.

Note: Even if you didn't get the entire derivation, you can still rule out "Never" as an answer choice, because in the example Gridworld, we see from Q4.5 and Q4.2 that the optimal policies computed are the same.

Q5 *RL: Biking in Berkeley*

(17 points)

Pacman is biking around campus, trying to find his classes. He models the campus as the Gridworld MDP shown to the right, where the transition probabilities and rewards are all unknown.

<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>
<i>E</i>	<i>F</i>	<i>G</i>	<i>H</i>
<i>I</i>	<i>J</i>	<i>K</i>	<i>L</i>

Remember that in Gridworld, actions don't always succeed. For example, if you move Right from *E*, you sometimes end up in *A* or *I*.

For Q5.1 to Q5.3, Pacman's policy π is to always move Right (except for states where he can only Exit and reach the terminal state End).

Q5.1 (3 points) For this subpart only, Pacman collects many samples, starting in state *E* each time and following π . In other words, in Pacman's samples (s, a, s', r) , state *s* is always *E*.

If you apply model-based learning, which values from the MDP can you estimate from the samples? Select all that apply.

☒ $\hat{T}(E, \text{Right}, A)$

☒ $\hat{T}(E, \text{Right}, I)$

☐ $\hat{T}(E, \text{Up}, I)$

☒ $\hat{T}(E, \text{Right}, F)$

☐ $\hat{T}(E, \text{Up}, A)$

☐ None of the above

Solution:

We can estimate any transition probability where the action is Right. For example, to estimate $T(E, \text{Right}, F)$, you'd count up the number of samples with (E, Right, F) and divide by the number of samples with (E, Right, s') for any successor state s' .

However, here is no way to estimate a transition probability where the action is Up, since we have no samples where the action is Up. The numerator and denominator in your count-based estimate would both be 0.

Q5.2 (3 points) For this subpart only, Pacman records one episode, and collects these four samples:

$(E, \text{Right}, F, 3)$ $(F, \text{Right}, J, 0)$ $(J, \text{Right}, K, 2)$ $(K, \text{Exit}, \text{End}, 100)$

If you apply direct evaluation, which values from the MDP can you estimate from the samples? Select all that apply.

☒ $V^\pi(E)$

☒ $V^\pi(F)$

☐ $V^\pi(G)$

☒ $V^\pi(J)$

☒ $V^\pi(K)$

☐ $V^\pi(L)$

Solution:

As long as Pacman has visited a state in the sample, we can compute the expected discounted reward from that state, according to the sample.

(Question 5 continued...)

Q5.3 (2 points) For this subpart only, Pacman records three episodes, and collects these 10 samples:

Episode (i):

- 1: (E , Right, F , 3)
- 2: (F , Right, J , 0)
- 3: (J , Right, K , 2)
- 4: (K , Exit, End, 100)

Episode (ii):

- 5: (C , Right, G , 5)
- 6: (G , Right, K , 10)
- 7: (K , Exit, End, 100)

Episode (iii):

- 8: (F , Right, G , 0)
- 9: (G , Right, K , 10)
- 10: (K , Exit, End, 100)

You apply TD learning (with $\alpha > 0, \gamma > 0$), processing the samples one at a time (from 1 to 10).

Reminder: In TD learning, $V^\pi(s)$ for all states starts at 0 before processing any samples.

Which sample causes $V^\pi(F)$ to become nonzero when that sample is processed?

☐ A Sample 1

☒ D Sample 8

☐ B Sample 2

☐ E Sample 10

☐ C Sample 4

☐ F $V^\pi(F)$ is 0 after processing all samples.

Solution: Sample 8. $V^\pi(F)$ is only updated after processing Samples 2 and 8. For Sample 2, $V^\pi(F)$ stays at zero because the immediate reward is 0 and $V^\pi(J)$ is 0. For Sample 8, $V^\pi(F)$ becomes nonzero because $\gamma V^\pi(G)$ will be nonzero by that time.

Subparts Q5.4 to Q5.6 are each independent.

(Question 5 continued...)

Q5.4 (4 points) Pacman follows policy π (move Right), and collects a finite number of samples. He runs Q-learning on these samples to compute a table Q_{init} (which has some nonzero entries).

Then, Pacman switches to a policy of picking a random action at each state. He collects many new samples, visiting each state-action pair infinitely often.

He tries processing the new samples in two different ways, using the same α for both runs:

- **Run A:** Initialize Q-values to Q_{init} , then run Q-learning with the new samples.
- **Run B:** Initialize Q-values to all zeroes, then run Q-learning with the new samples.

Select all true statements.

- ☒ **A** If α is decreased to 0 over time, both runs can converge to the same optimal Q^* .
- ☐ **B** With a constant and nonzero α , both runs are guaranteed to converge to fixed Q-values.
- ☐ **C** Since Run A begins with nonzero Q-values, Run A will always converge faster than Run B.
- ☒ **D** If α is decreased to 0 over time, then the learned policies in the runs can differ.
- ☐ **E** None of the above

Solution:

1. Since we visit every (s,a) pair infinitely many times, we can converge to Q^* given an appropriate learning rate that decreases to 0 over time.
2. With a constant learning rate, we could have fluctuating Q-values that do not converge.
3. There is no guarantee of faster convergence based on Q-value initialization.
4. If α decreases too fast, the Q-values can be inaccurate, resulting in differing policies.

(Question 5 continued...)

Q5.5 (2 points) Pacman runs approximate Q-learning, with two features:

$$Q(s, a) = w_1 f_1(s, a) + w_2 f_2(s, a)$$

- $f_1(s, a)$ represents *hilliness*, a number between 0 (flat) and 1 (steep).
- $f_2(s, a)$ represents *crowd size*, a number between 0 (empty) and 1 (crowded).

After several iterations, Pacman has learned weights $w_1 = 1$ and $w_2 = -5$.

Pacman acts exploitatively (maximizing expected reward) based on these weights.

Select all true statements.

- ☐ A If choosing between two Q-states with equal crowd sizes, Pacman prefers the flatter Q-state.
- ☒ B If choosing between two Q-states with equal hilliness, Pacman prefers the emptier Q-state.
- ☐ C None of the above

Solution: When Pacman encounters steeper hills (higher values for hilliness), this term adds more to Q, making the action more appealing. Pacman prefers steeper paths because they yield higher Q-values, making choice B correct. When Pacman encounters more crowded paths (higher values for crowd size), this term subtracts more from Q, making the action less appealing. Thus, Pacman prefers less crowded paths because they yield higher Q-values, making choice C correct. Finally, the magnitude of $w_2 > w_1$, so crowd size has a greater effect on Pacman's decision-making, making choice F correct.

Q5.6 (3 points) In general (not necessarily for any specific MDP), when is following an ϵ -greedy strategy with $\epsilon > 0$ useful, compared to $\epsilon = 0$? Select all that apply.

- ☒ A When not all states have been visited.
- ☒ B When all states have been visited exactly once.
- ☐ C When the Q-values have converged to the optimal Q-values.
- ☐ D None of the above

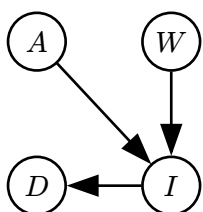
Solution: Exploration is helpful when we have not converged to the optimal Q-values. If we have not visited all states or only visited states once, we likely have not converged to the optimal Q-values.

Q6 Bayes Nets: Probabilistic Planes

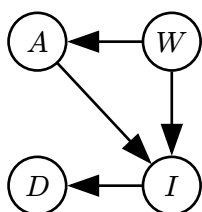
(10 points)

We want to model airplane departures based on several factors. Air traffic (A) and weather (W) affect inbound flight arrivals (I), and I affects the departure time (D).

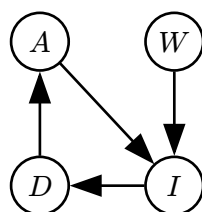
Q6.1 (2 points) Select all of the well-formed Bayes Nets that *could* represent a scenario consistent with the assertions given above (even if the Bayes Net is not the most efficient representation).



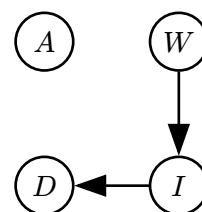
☒ Bayes Net (i)



☒ Bayes Net (ii)



☐ Bayes Net (iii)



☐ Bayes Net (iv)

Solution:

A is correct because we know that A and W both affect I , and I affects D . We have arrows demonstrating all of these relationships.

B is correct for similar reasons as A. However, B now contains an arrow from W to A . This is still a valid representation (though not the most efficient) because it remains a directed acyclic graph, and the additional arrow does not introduce any contradictions.

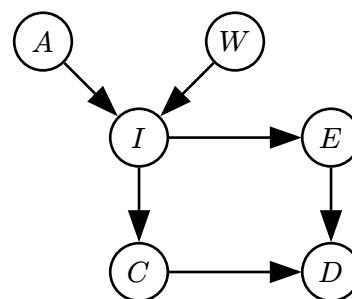
C is incorrect as we have a cycle, which is invalid in Bayes Net representations.

D is incorrect because we know A affects I , but we are missing an arrow to demonstrate this relationship.

For the rest of the question, consider the Bayes Net to the right, with added variables E and C . Assume that all variables are ternary (i.e. each variable has a domain size of 3).

Q6.2 (2 points) How many entries are in I 's CPT (conditional probability table)?

- ☒ 3 ☐ 9 ☐ 27 ☐ 81



Solution: We would need $P(I \mid A, W)$. Since there are 3 ternary variables, there are $3^3 = 27$ entries in the CPT for I .

For Q6.3 to Q6.6, use d -separation to determine if the independence assumption is true or false.

Q6.3 (1 point) $E \perp\!\!\!\perp C$ ☐ True ☒ False

Q6.4 (1 point) $W \perp\!\!\!\perp D \mid I$ ☒ True ☐ False

(Question 6 continued...)

Q6.5 (1 point) $A \perp\!\!\!\perp W \mid D$

☐ True

☒ False

Q6.6 (1 point) $A \perp\!\!\!\perp C \mid E$

☐ True

☒ False

Solution:

$E \perp\!\!\!\perp C$ is false.

$C \leftarrow I \rightarrow E$ forms a common cause triple, so C and E are not independent.

$W \perp\!\!\!\perp D \mid I$ is true.

There is no active path from W to D given I since both $W \rightarrow I \rightarrow E$ and $W \rightarrow I \rightarrow C$ are both causal chains.

$A \perp\!\!\!\perp W \mid D$ is false.

$A \rightarrow I \leftarrow W$ forms a common effect triple with D as a descendant of I , so A and W are not independent.

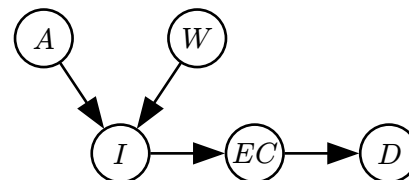
$A \perp\!\!\!\perp C \mid E$ is false.

There are two undirected paths from A to C . The triple $A \rightarrow I \rightarrow C$ is an active triple, making this an active path, so A and C are not independent.

Q6.7 (2 points) Suppose we combine E and C into one variable EC , as shown below.

The values that EC takes on are tuples of (E, C) , representing all combinations of E 's values and C 's values from the original Bayes Net.

How does the total number of entries across all CPTs change, compared to the original Bayes Net?



☒ Increases by 9 entries

☐ Decreases by 9 entries

☐ Increases by 27 entries

☐ Decreases by 27 entries

Solution: Looking at the CPTs that differ between the two representations, we replace the tables $P(E \mid I)$, $P(C \mid I)$, $P(D \mid C, E)$ with $P(EC \mid I)$ and $P(D \mid EC)$. The initial tables have $3^2 + 3^2 + 3^3 = 45$ total entries. Since EC represents all combinations of E and C , its domain has 9 values. The new tables have $9 \cdot 3 + 3 \cdot 9 = 54$ entries, which is an increase of 9 entries.